

TBA4251 - PROGRAMMERING I GEOMATIKK

GISc oppgave - *MyGIS*

Av:
Johanne Holmøy

05.12.2025

Innhold

Figurer	ii
1 Introduksjon	1
2 Bakgrunn og formål	1
3 Teknologier og datasett	1
3.1 Systemarkitektur og rammeverk	1
3.2 Kartmotor og beregninger	2
3.3 Datasett og format	2
4 Arbeidsgang	2
5 Brukerflyt og bruk av applikasjonen	3
5.1 Startskjerm	3
5.2 Oppgaveoversikt	4
5.3 Lagoversikt og kontroll	5
5.4 Tilbakemeldinger og feilhåndtering	5
5.5 Fullført oppgavegjennomgangen	7
6 Diskusjon	7
6.1 Valg av kartmotor	7
6.2 Separasjon av beregning og grensesnitt	8
6.3 Datagrunnlag og begrensninger	8
6.4 Manglende backend	8
6.5 Brukeropplevelse	8
7 Forbedringspotensiale	8
7.1 Ytelse ved tunge operasjoner	8
7.2 Mer detaljert bygningsdata	9
7.3 Backend for lagring og deling	9
8 Konklusjon	9
Appendix	10
A Programstruktur	10
B Referanseliste	11
C Oppgavetekster	11

Figurer

1	Startskjermen ved oppstart av applikasjonen.	4
2	Verktøygjennomgangen forklarer funksjonaliteten til alle verktøyene	4
3	Oppgavene presenteres i egen popup med frem- og tilbake-navigasjon.	5
4	Kartvisning og lagoversikt etter opplasting av data.	5
5	Feilmelding ved ugyldig filtrering.	6
6	Verktøy er deaktivert frem til minst ett lag er lastet opp.	6
7	Dropdown-menyer er brukt for å begrense muligheten for brukerfeil.	7
8	Vinduet som vises når alle oppgavene er gjennomført.	7

1 Introduksjon

Denne rapporten presenterer utviklingen av en webbasert GIS-applikasjon i henhold til oppgaven gitt i emnet TBA4251 Programmering i Geomatikk. Oppgaven besto av å utforme et lite GIS som inneholder viktig GIS funksjonalitet, og applikasjonen skal illustrere sentrale geodataoperasjoner på en pedagogisk og anvendbar måte.

Applikasjonen danner et scenario hvor Studentskipnaden i Trondheim (SiT) ber brukeren finne egnede områder i Trondheim til utbygging av nye studentboliger. Brukeren guides igjennom en romlig analyse ved hjelp av beskrivende oppgaver som anvender ulike GIS-verktøy. Analysen tar utgangspunkt i reelle geografiske data, og gir brukeren en innsikt i hvordan GIS kan benyttes som beslutningsverktøy i arealplanlegging.

Applikasjonen er tilgjengelig på følgende lenke:

<https://johholmo.github.io/TBA4251-Programmering-i-geomatikk/>

2 Bakgrunn og formål

Bakgrunn for applikasjonen er å illustrere for brukeren hvordan GIS kan brukes til å bestemme hvor det er både mulig og hensiktsmessig å bygge studentboliger. Studentene som skal anvende applikasjonen er bosatt i Trondheim og er kjent med både Trondheim og studentboliger, og dermed er problemstillingen enkel å forstå for brukeren.

Formålet med applikasjonen er todelt. Det er et pedagogisk formål å gi nye GIS-brukere en enkel og praktisk innføring i grunnleggende romlige analyser, uten krav til forkunnskaper eller installasjon av avansert programvare. Oppgavene i applikasjonen er presentert steg for steg, slik at brukeren blir guidet gjennom datasett og verktøy mot en komplett analyse.

Det faglige formålet med applikasjonen er å gjennomføre en romlig analyse ved hjelp av et GIS-verktøy, for å finne hvilke områder i Trondheim som kan egne seg for utbygging av studentboliger. I analysen vurderes blant annet trygg avstand til vann, beliggenhet i forhold til campus, og hvor store tomtene må være for å kunne bygges ut. Applikasjonen bruker vanlige GIS-operasjoner slik som klipp, buffer, differanse og intersect for å snevre inn fra hele Trondheim kommun til de egnede byggeområdene.

Kort sagt skal applikasjonen både lære bort hvordan en romlig analyse gjennomføres i praksis og samtidig vise et konkret resultat som gir mening for studenter i Trondheim.

3 Teknologier og datasett

En full oversikt over fil- og mappeoppsett finnes i Vedlegg A. Der finnes også en oversikt over anvendt dokumentasjon, kilder og tekniske referanser (Vedlegg B).

3.1 Systemarkitektur og rammeverk

Applikasjonen er utviklet som en klientside webapplikasjon i *React* og *TypeScript*. React brukes til å bygge og organisere grensesnittet, mens TypeScript gir tryggere håndtering av data og romlige objekter. Hver GIS-funksjon, slik som **Clip**, **Buffer** og **Intersect**, er laget som egne verktøy. Både verktøy og oppgaver åpnes i en enkel popup, slik at brukeren kun forholder seg til ett steg av gangen.

Kartlagene administreres gjennom en egen **LayersContext**, der brukeren kan skru av og på lag, endre navn og farge, og slette eller reorderere dem. Dette illustreres i en oversikt til venstre i dashbordet.

Applikasjonen bruker et melding- og varslingssystem (`CustomToastProvider`, spinnere, og diverse errormeldinger) som gir tilbakemeldinger ved bruk av verktøy, enten under utførelse eller ved oppståtte feil.

3.2 Kartmotor og beregninger

Kartet vises med *Mapbox GL JS*, mens *Mapbox Draw* blir anvendt når brukeren skal tegne sitt eget analyseområde. All kartdata vises som GeoJSON i WGS 84 (EPSG:4326), som er standard for Mapbox.

De tunge romlige operasjonene, som buffer, intersect og union, kjøres i web workers, slik at den tunge beregningen tilhørende verktøyet skjer utenfor hovedtråden i nettleseren. Dermed vil ikke UI fryse for brukeren mens analyser pågår. Arbeidsdelingen håndteres av `geoworkerClient` som kommuniserer mellom UI og `geoworkers`.

En rekke støttefunksjoner finnes i `geomaticFunctions` og `commonFunctions`, og brukes blant annet til opprydding av geometri, fargevalg og enkel kontroll av egenskaper i datasettet. Her finnes spesielt funksjoner for å bestemme geometrityper (`isPoly`, `isCoordGeometry`), transformere koordinater (`toWGS84`, `transformGeometry`), og diverse geometriske verktøy (`unionPolygons`, `explodeToPolygons`).

3.3 Datasett og format

Applikasjonen bruker lokale datasett for Trondheim i GeoJSON-format. Alle filer, med unntak av **NTNU-campus**, er hentet fra GeoNorge og konvertert fra GML til GeoJSON før bruk for å gjøre dem lettere å håndtere i nettleseren.

Følgende datasett brukes i analysen:

- **Trondheim kommunegrense** – setter ytterkant for analysen.
- **Vei** – brukes for å vurdere tilgjengelighet og sentralitet.
- **Kanal, elv, innsjø og havflate** – slås sammen til et eget *Vannlag* som brukes for å lage buffere rundt vann og hindre bygging for nær vannkanten.
- **Bygning** – brukes for å filtrere bort uaktuelle områder.
- **NTNU-campus** – punkter laget manuelt med koordinater fra Google Maps, brukt til bufferanalyse rundt studiestedene.

Opplasting gjøres med `Upload`-verktøyet, som også sjekker og tilpasser koordinatsystemet. Dersom et datasett ikke er i WGS 84, reprojiseres det automatisk fra EPSG:25832. Alle datasett blir dermed standardisert og kan brukes i de samme analyseoperasjonene uten videre konvertering.

Resultat fra bruk av verktøy lagres som nye GeoJSON-lag med egne farger og navn, slik at brukeren enkelt kan sammenligne før- og etter uten å ødelegge originaldataene.

4 Arbeidsgang

Arbeidet startet med å definere hva applikasjonen skulle brukes til, hvem den var laget for, og hvilke GIS-funksjoner som måtte være tilgjengelige. Siden målgruppen er brukere uten tidligere erfaring med GIS, ble det tidlig bestemt å lage et enkelt grensesnitt med kart, lagoversikt og et begrenset sett grunnleggende verktøy.

I første fase ble design og oppsett av nettsiden, samt følgende kjerneverktøy utviklet: **Upload**, **Clip**, **Buffer**, **Union**, **Difference** og **Intersect**. Disse verktøyene var nødvendige for applikasjonen, og gjorde det enklere å danne oppgavestrukturen i senere fase. Samtidig ble visning av data i GeoJSON-format og laghåndtering via **LayersContext** etablert, slik at brukeren kunne se datalagene i kartet og tilpasse dem på en oversiktlig måte.

Videre ble oppgavene bygget inn i applikasjonen. Oppgavene følger en fast rekkefølge der hver oppgave bygger videre på resultatet fra den forrige. Underveis i prosessen kom det fram at applikasjonen hadde behov for flere verktøy enn først planlagt, og dermed ble **FeatureExtractor** og **AreaFilter** utviklet. Videre viste testing med større datasett ulike ytelsesproblemer, spesielt forsinkelser og i enkelte tilfeller krasj av nettsiden.

For å løse ytelsesproblemene ble kartmotoren byttet fra Leaflet til Mapbox, som håndterer vektor-data raskere og mer stabilt. I tillegg ble alle tunge romlige operasjoner flyttet fra verktøyene og ut i egne web workers. Dette gjorde at analysene kunne kjøre i bakgrunnen uten å fryse brukergrensesnittet.

Til slutt ble koordinathåndteringen forenklet. All transformasjon blir gjort ved innlasting av data, slik at videre operasjoner kan utføres uten nye konverteringer. Dette var en opprydning av en stegvis utvikling, som førte til en mer stabil og effektiv analyseflyt. Dermed kan brukeren gjennomføre hele prosessen direkte i nettleseren uten spesielt merkbare forsinkelser.

5 Brukerflyt og bruk av applikasjonen

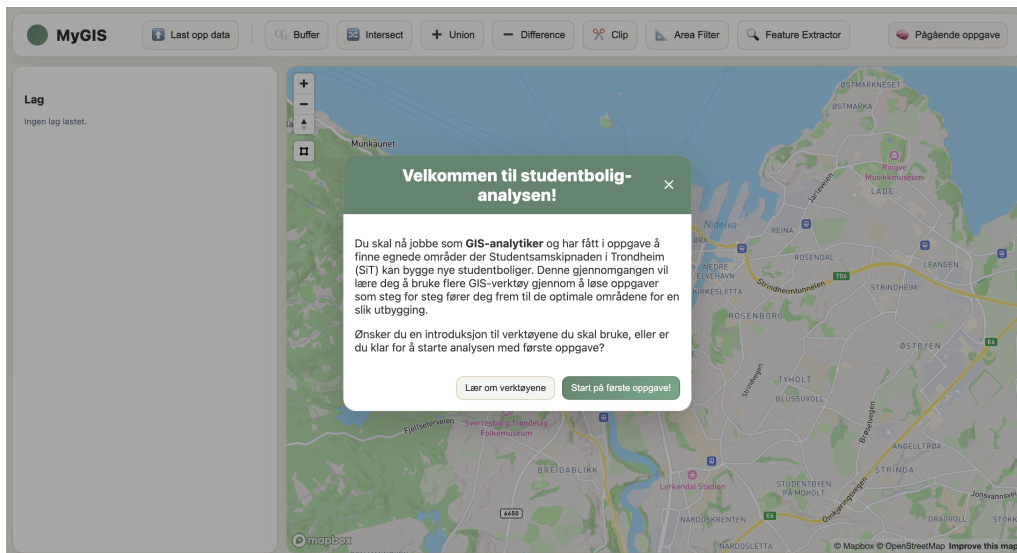
Denne seksjonen beskriver hvordan brukeren beveger seg gjennom applikasjonen og fullfører analysen. Ettersom oppgavene er presentert som en gjennomgang i applikasjonen, vil de ikke bli presentert i detalj i denne rapporten. En oversikt over alle oppgavetekster fra applikasjonen kan derimot finnes i Vedlegg C.

Applikasjonen er bygget som en lineær oppgaveflyt, og brukeren trenger ingen GIS-erfaring for å gjennomføre stegene. Hvert verktøy og hvert datasett forklares i popup-format, slik at brukeren kan fokusere på en handling av gangen.

5.1 Startskjerm

Når applikasjonen åpnes får brukeren en kort introduksjon og kan velge å starte på oppgavene eller gå igjennom en enkel gjennomgang av verktøyene. Brukeren får også sett selve dashbordet som ligger bak popuen.

Velkomstskjermen er vist i Figur 1, mens Figur 2 viser første steg i verktøygjennomgangen.



Figur 1: Startskjermen ved oppstart av applikasjonen.



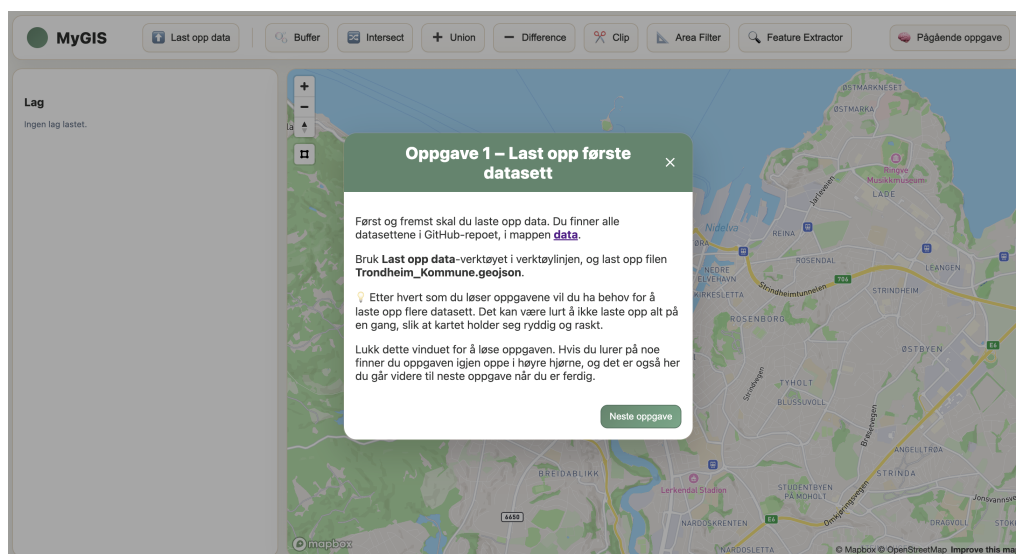
Figur 2: Verktøygjennomgangen forklarer funksjonaliteten til alle verktøyene

5.2 Oppgaveoversikt

Applikasjonen består av elleve oppgaver som presenteres etter tur. Hver oppgave anvender ett eller flere verktøy, og produserer et nytt resultatlag som brukes videre:

1. Last opp data, ved bruk av *Upload*
2. Tegn analyseområde, ved bruk av *Map-draw*
3. Klipp til analyseområdet, ved bruk av *Clip*
4. Buffer rundt vann, ved bruk av *Buffer*
5. Fjern områder nær, vann ved bruk av *Difference*
6. Filtreer bygninger og hensyn, ved bruk av *FeatureExtractor*
7. Buffer rundt campus, ved bruk av *Buffer*
8. Finn overlappende områder, ved bruk av *Intersect*
9. Kombiner aktuelle områder, ved bruk av *Union/Intersect*
10. Velg store nok tomter, ved bruk av *Area Filter*
11. Resultatlaget vises

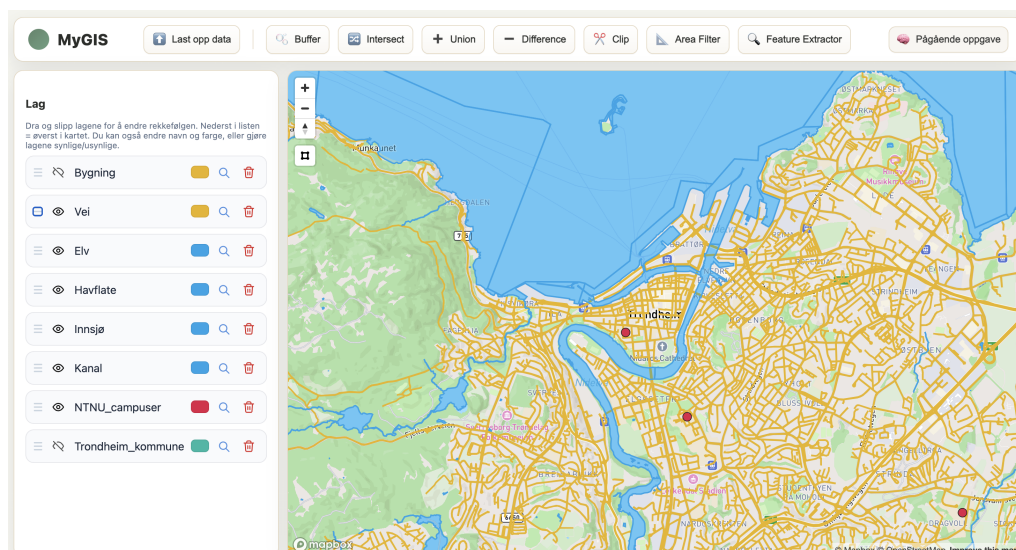
Oppgavene presenteres i et popup-vindu (Figur 3), og leder brukeren igjennom arbeidsflyten. Hver oppgave gir brukeren tydelig informasjon om hvilket datasett og verktøy som skal brukes.



Figur 3: Oppgavene presenteres i egen popup med frem- og tilbake-navigasjon.

5.3 Lagoversikt og kontroll

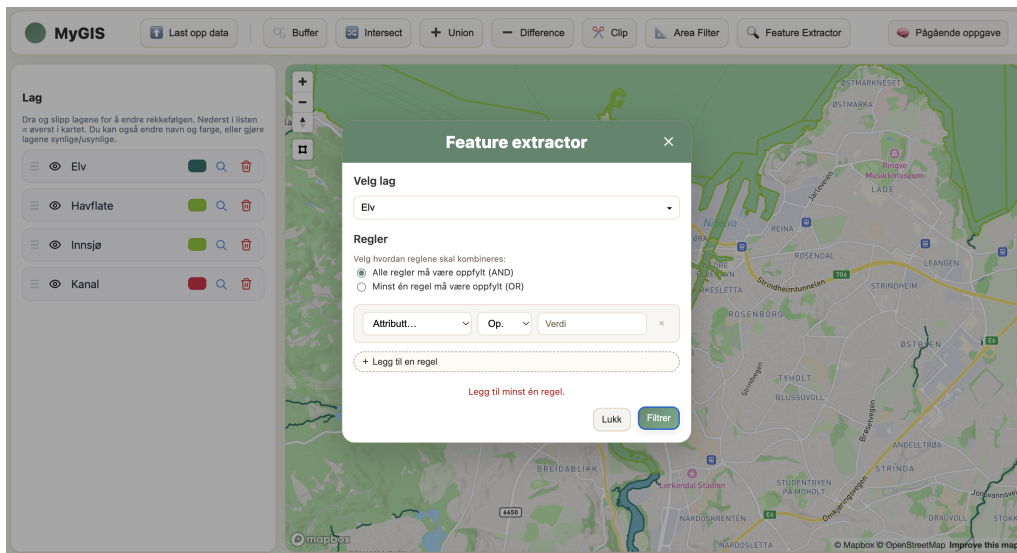
Alle lag som opprettes ved bruk av et verktøy, eller lastes opp, havner i en enkel oversikt til venstre i dahsbordet (Figur 4). Her kan brukeren vise/skjule, endre navn og farge, eller slette lag. Brukeren kan dermed aktivt vurdere hvilke lag som er aktuelle å vise i kartet, samt styre visningen av disse.



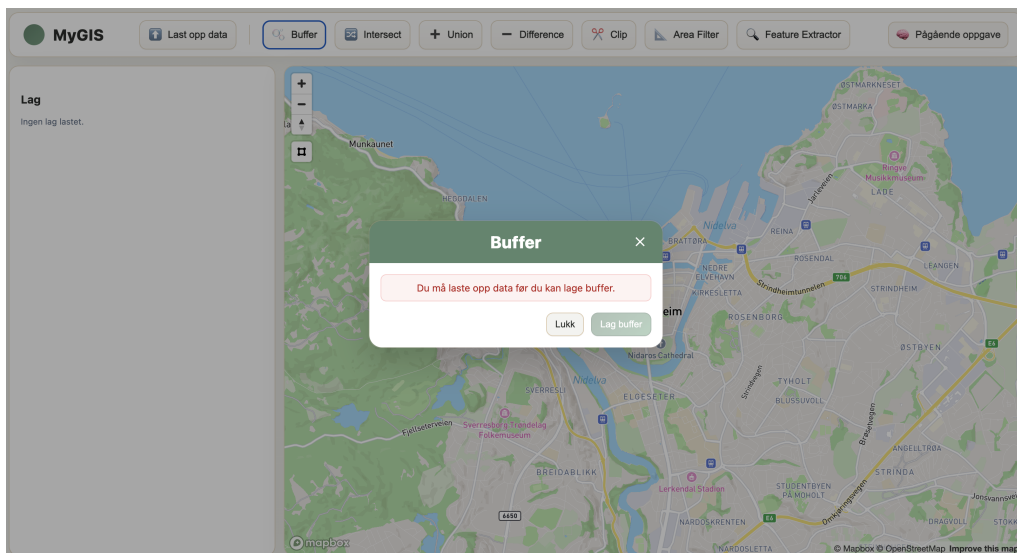
Figur 4: Kartvisning og lagoversikt etter opplasting av data.

5.4 Tilbakemeldinger og feilhåndtering

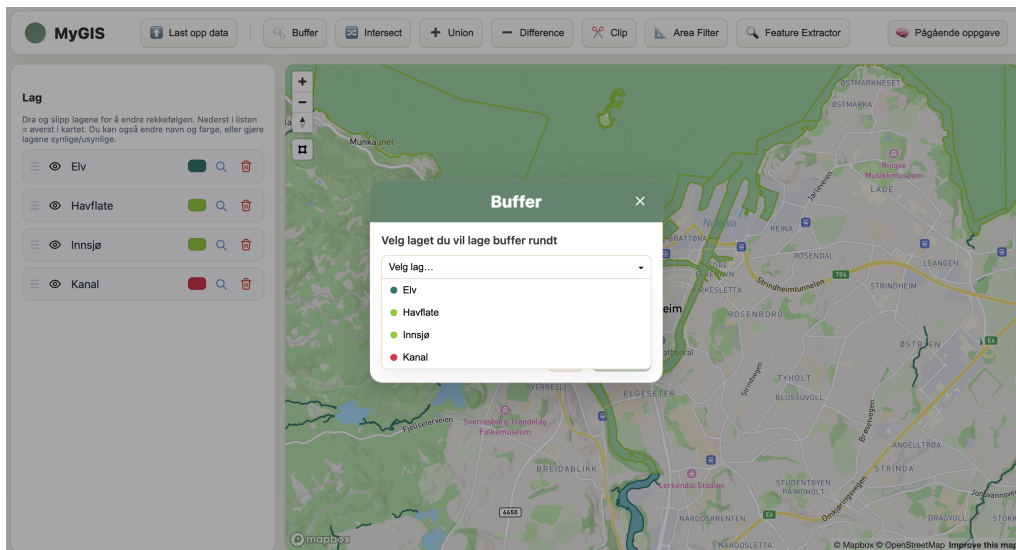
Applikasjonen vil gi varslinger og tilbakemeldinger til brukeren om en operasjon ikke kan gjennomføres eller blir forsøkt gjennomført på feil måte (Figur 5 og 6). I tillegg er dropdown-menyer for valg av lag og attributter implementert på flere av verktøyene for å begrense muligheten for brukerfeil (Figur 7).



Figur 5: Feilmelding ved ugyldig filtrering.



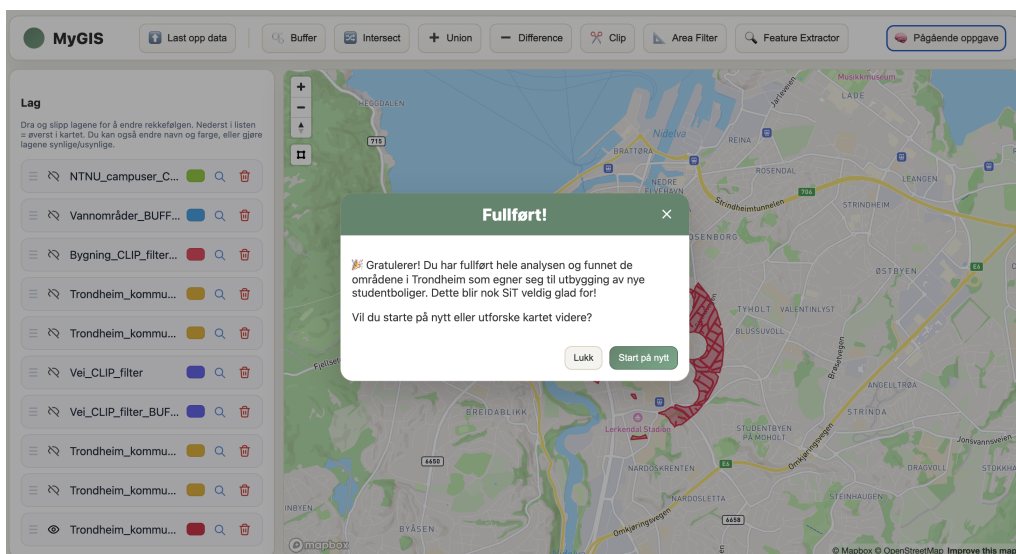
Figur 6: Verktøy er deaktivert frem til minst ett lag er lastet opp.



Figur 7: Dropdown-menyer er brukt for å begrense muligheten for brukerfeil.

5.5 Fullført oppgavegjennomgangen

Når siste oppgave er ferdig, får brukeren et *Fullført*-vindu (Figur 8) og vil ha laget et endelig resultatlag med mulige områder for utbygging av studentboliger. Brukeren får da valget om å fortsette å utforske kartet eller starte analysen på nytt. Starter brukeren analysen på nytt vil alle lagene slettes, og brukeren får presentert velkomstmeldingen igjen.



Figur 8: Vinduet som vises når alle oppgavene er gjennomført.

6 Diskusjon

6.1 Valg av kartmotor

Applikasjonen ble først utviklet i Leaflet, men buffer og differanse på større linje- og polygondata førte til treghet og krasj av nettsiden. Ved å bytte til Mapbox GL fikk applikasjonen en mer stabil

og raskere håndtering av GeoJSON-data, og viste seg derfor å være et bedre valg for denne typen analyse.

6.2 Separasjon av beregning og grensesnitt

Alle romlige operasjoner ble først kjørt direkte i hovedtråden i nettleseren, som førte til at nettsiden frøs og kunne krasje når større analyser ble gjort. Ved å flytte beregningene til egne web workers ble analysene gjort i bakgrunnen, mens brukeren blir presentert en spinner for å indikere at applikasjonen arbeider.

6.3 Datagrunnlag og begrensninger

Datasett fra GeoNorge fungerte godt, men i utgangspunktet ble det brukt bygningsdata i linjeformat (Taklinjer), som et bygningsomriss. Buffering på disse linjene ga svært mange detaljer og dermed tunge beregninger. Dette var hovedgrunnen til at å fjerne alle bygninger med buffer fra mulige byggeområder, ikke ble en del av sluttløsningen. Det er derimot ikke kritisk i en slik analyse da det er mulig å rive bygninger for å bygge nye, men det hadde vært ønsket å inkludere. Erfaringen viser at kvaliteten og strukturen på datasettet påvirker hvor egnet det er for sanntidsanalyse i nettleser.

6.4 Manglende backend

Fravær av backend gjør applikasjonen enkel å drifte og lett å bruke, men noe begrenser funksjonaliteten. Det er ikke mulig å lagre prosjekt, dele analyser eller hente tilbake tidligere resultat. For opplæringsformål fungerer løsningen likevel godt, da det ikke vil være nødvendig med slik funksjonalitet.

6.5 Brukeropplevelse

En popup-basert oppgaveflyt gjør det tydelig for brukeren hva som skal gjøres i hvert steg, og hindrer at for mange valg vises samtidig. Verktøyene er deaktivert frem til nødvendige lag er lastet opp, noe som både begrenser feil og gjør det enklere å følge analysen i riktig rekkefølge. Lagoversikten gir god kontroll over hvilke områder som er aktive, og hvilke som har vært laget tidligere.

Oppsummert ungerer applikasjonen slik den var planlagt: som et enkelt og pedagogisk verktøy for å lære grunnleggende lokaliseringsanalyse. Applikasjonen lar en ny GIS-bruker fint gjennomføre hele analysen uten forkunnskap og uten behov for tradisjonell GIS-programvare.

7 Forbedringspotensiale

7.1 Ytelse ved tunge operasjoner

Til tross for at web-workers avlaster tunge beregninger, er det fremdeles rom for forbedring av hastighet ved bruk av union og difference på store datasett. Dette gjelder spesielt når mange små polygoner skal slås sammen, eller når store datasett av linjer skal trekkes fra et polygon-lag (for eksempel trekke veidata fra Trondheim kommune). En naturlig videreutvikling av applikasjonen er å implementere et *dissolve*-verktøy, slik at overlappende flater kan samles til én enhet. Dette kan både redusere beregningstid og gjøre kartvisningen ryddigere.

7.2 Mer detaljert bygningsdata

Jeg presterte ikke å få tak i bygningsdata i form av polygoner, og ettersom bygningsomriss i form av linjesegmenter (taklinjer) skapte for tung geometri og redusert ytelse, ble det heller anvendt punktdata i applikasjonen og oppgavene. Det er derfor et naturlig forbedringspotensiale å legge inn bygningspolygoner, slik at analysen kan utføres mer presist og eksisterende bygninger kan fjernes fra aktuelle byggeområder.

7.3 Backend for lagring og deling

Applikasjonen fungerer godt som et rent nettleserbasert verktøy, men en enkel backend vil åpne for muligheten å lagre prosjekter og ta opp arbeidet senere, eksportere sluttresultat som GeoJSON, PDF eller kartbilde, og dele analyser mellom studenter. Dette er ikke nødvendig for formålet med applikasjonen, men kan være en spennende videreutvikling.

8 Konklusjon

Målet med prosjektet var å utvikle en webbasert GIS-applikasjon som på en pedagogisk og visuell måte demonstrerer romlig analyse gjennom et stegvis oppgavesett. Applikasjonen oppfylder dette målet ved at en bruker uten tidligere GIS-kunnskaper kan gjennomføre en full analyse, fra rådata til identifisering av egnede byggeområder i Trondheim.

Stabiliteten i applikasjonen oppnås ved bruk av Mapbox for vektorrendering og web-workers for tunge geometrier, samt kontrollerte popups og et ryddig dashboard. Oppgavestrukturen gir en naturlig progresjon og fungerer som introduksjonsverktøy til GIS.

Det gjenstår fremdeles potensial for videreutvikling, spesielt ved støtte for mer komplekse datasett, back-endlagring og flere analyseoperasjoner. Likevel er løsningen en robust og nybegynnervennlig GIS-plattform som oppfylder kravene i prosjektet og kan være godt egnet som læringsverktøy for kommende geomatikkstudenter.

Vedlegg

A Programstruktur

```
.
|-- README.md
|-- data
| |-- Bygning.geojson
| |-- Elv.geojson
| |-- Havflate.geojson
| |-- Innsjø.geojson
| |-- Kanal.geojson
| |-- NTNU_campuser.geojson
| |-- Trondheim_kommune.geojson
| `-- Vei.geojson
|-- eslint.config.js
|-- index.html
|-- package-lock.json
|-- package.json
|-- public
| `-- vite.svg
|-- src
| |-- App.tsx
| |-- assets
| | `-- react.svg
| |-- components
| | |-- Navbar.tsx
| | |-- Sidebar.tsx
| | |-- ToolTour.tsx
| | |-- ToolbarButton.tsx
| | |-- map
| | | `-- Map.tsx
| | |-- popup
| | | |-- NamingPopup.tsx
| | | `-- Popup.tsx
| | |-- tasks
| | | |-- Done.tsx
| | | |-- Task1.tsx
| | | |-- Task2.tsx
| | | |-- Task3.tsx
| | | |-- Task4.tsx
| | | |-- Task5.tsx
| | | |-- Task6.tsx
| | | |-- Task7.tsx
| | | |-- Task8.tsx
| | | |-- Task9.tsx
| | | |-- Task10.tsx
| | | |-- Task11.tsx
| | | `-- Welcome.tsx
| | `-- tools
| | |-- AreaFilter.tsx
| | |-- Buffer.tsx
| | |-- Clip.tsx
| | |-- Difference.tsx
| | |-- FeatureExtractor.tsx
| | |-- Intersect.tsx
| | |-- Union.tsx
```

```
| | `-- Upload.tsx
| |-- context
| | `-- LayersContext.tsx
| |-- index.css
| |-- main.tsx
| |-- stores
| | `-- layers.ts
| |-- styles.css
| |-- types
| | `-- reproject.d.ts
| |-- utils
| | |-- CustomToastProvider.tsx
| | |-- commonFunctions.ts
| | |-- geomaticFunctions.ts
| | |-- icons.tsx
| | `-- toasts.ts
| `-- workers
| |-- geoworkerClient.ts
| `-- geoworkers.ts
|-- tsconfig.app.json
|-- tsconfig.json
|-- tsconfig.node.json
`-- vite.config.ts
```

B Referanseliste

GeoNorge. *Nasjonal Geodatasettportal*. Tilgjengelig fra: <https://www.geonorge.no>. Hentet september/oktober 2025.

Mapbox. *Mapbox GL JS – Developer Documentation*. Tilgjengelig fra: <https://docs.mapbox.com/mapbox-gl-js>. Hentet september 2025.

Turf.js. *Geospatial Analysis Library for JavaScript*. Tilgjengelig fra: <https://turfjs.org>. Hentet september 2025.

EPSG Geodetic Parameter Dataset. *European Petroleum Survey Group*. Tilgjengelig fra: <https://epsg.io/>. Hentet september 2025.

C Oppgavetekster

Velkommen

Du skal nå jobbe som **GIS-analytiker** og har fått i oppgave å finne egnede områder der Studentsamskipnaden i Trondheim (SiT) kan bygge nye studentboliger. Denne gjennomgangen vil lære deg å bruke flere GIS-verktøy gjennom å løse oppgaver som steg for steg fører deg frem til de optimale områdene for en slik utbygging.

Ønsker du en introduksjon til verktøyene du skal bruke, eller er du klar for å starte analysen med første oppgave?

Oppgave 1 – Last opp første datasett

Først og fremst skal du laste opp data. Du finner alle datasettene i GitHub-repoet, i mappen **data**.

Bruk **Last opp data**-verktøyet i verktøylinjen, og last opp filen **Trondheim_Kommune.geojson**.

Etter hvert som du løser oppgavene vil du ha behov for å laste opp flere datasett. Det kan være lurt å ikke laste opp alt på en gang, slik at kartet holder seg ryddig og raskt.

Lukk dette vinduet for å løse oppgaven. Hvis du lur på noe finner du oppgaven igjen oppe i høyre hjørne, og det er også her du går videre til neste oppgave når du er ferdig.

Oppgave 2 – Lag et Area of Interest (AOI)

I kartet ser du nå et polygon som dekker hele Trondheim Kommune. Det er ikke nødvendig å analysere et så stort område, så du skal nå begrense analysen til et mindre område mer aktuelt for studentboliger.

I kartet (rett under zoom-knappene) har du en knapp som lar deg tegne et polygon. Bruk denne til å lage et **Area of Interest (AOI)**.

Klikk deg rundt området for å lage polygonet, og avslutt ved å klikke på første punkt igjen. Pass på å lage et stort nok område.

Du kan gå videre til neste oppgave når du har laget ditt AOI og det er dukket opp i oversikten til venstre.

Oppgave 3 – Klipp data til AOI

Du skal nå klippe Trondheim_Kommune til polygonlaget du tegnet i forrige oppgave (Area of Interest), slik at du kun forholder deg til data innenfor det aktuelle området i kartet. Dette vil du gjøre med alle datalag du laster opp.

Åpne **Clip**-verktøyet i verktøylinjen. Laget som skal klippes er Trondheim_kommune, og det klippes mot Area of Interest.

Når du har utført klippingen vil du se et nytt datalag i oversikten til venstre. Dette er delen av Trondheim Kommune som ligger innenfor polygonet du tegnet. Gi dette laget et tydelig navn, for eksempel **AOI_Trondheim**, og gjerne endre fargen. Du kan da slette det opprinnelige Trondheim_Kommune-laget, men behold polygonlaget du tegnet slik at du kan klippe andre datalag mot dette senere.

Oppgave 4 – Vannområder

Først og fremst skal du sørge for at SiT ikke bygger nye studentboliger i vannområder. Last opp datasettene **Innsjø**, **Kanal**, **Elv** og **Havflate**, og klipp dem til AOI. Slett de originale lagene.

Alle disse datalagene representerer vann, så hvorfor ikke slå dem sammen til ett lag? Bruk **Union**-verktøyet for å slå sammen de klippede datalagene for vann til ett lag. Navngi og fargelegg laget passende, for eksempel *Vannområder*. Slett de individuelle lagene.

Oppgave 5 – Buffer

For å være på den sikre siden vil ikke SiT bygge så nærme vannområder heller. Du skal derfor lage en buffersone rundt vannområdene det ikke skal bygges i.

Bruk **Buffer**-verktøyet og lag en buffer på **200 meter** rundt vannområdene.

Oppgave 6 – Fjern uaktuelle områder

Du har nå laget en buffersone rundt vannområdene, som det ikke skal bygges i. Derfor skal du fjerne disse områdene fra **AOI_Trondheim**.

Bruk **Difference**-verktøyet, og trekk buffersonen fra **AOI_Trondheim**.

Det nye laget viser områder som fortsatt kan være aktuelle å bygge i. Du kan slette det opprinnelige **AOI_Trondheim**-laget og navngi det nye, for eksempel *Mulige byggeområder*.

Oppgave 7 – Unngå spesifikke bygninger

SiT vil gjerne unngå å rive universitetsbygg og offentlige bygg for barn og unge.

Last opp **Bygninger**, klipp det til AOI, og bruk **Feature Extractor** til å filtrere bygninger der bygningstype er mellom 600 og 699.

Lag en buffer på **150 meter** rundt disse bygningene, og fjern dem fra *Mulige byggeområder*.

Oppgave 8 – Unngå veier

Last opp datasettet **Vei** og klipp det til AOI. Bruk **Difference** for å fjerne veiene fra mulige byggeområder.

Deretter: bruk **Feature Extractor** for å finne veier der vegkategori er E eller R. Lag en buffer på **300 meter** og trekk den fra byggeområdene. Navngi laget *Mulige byggeområder*.

Oppgave 9 – I nærheten av NTNU

SiT ønsker at studentboliger skal ligge innenfor **1 kilometer** fra campus.

Last opp **NTNU_campus**, bruk **Buffer** på **1000 meter** og navngi laget *Ønskede bygningsområder*.

Oppgave 10 – Slå sammen trygge og ønskede områder

Du må finne områdene som både er trygge og ønskelige.

Bruk **Intersect** mellom *Mulige byggeområder* og *Ønskede bygningsområder*.

Navngi laget *Trygge og ønskede områder*.

Oppgave 11 – Store nok områder

Området må være stort nok for studentboliger. SiT ønsker minst **800 m²**.

Bruk **Area Filter** til å finne områder over 800 m².

Fullført!

Gratulerer! Du har fullført hele analysen og funnet de områdene i Trondheim som egner seg til utbygging av nye studentboliger.

Vil du starte på nytt eller utforske kartet videre?